

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий
Дисциплина «Естественно-языковой интерфейс интеллектуальных систем»

ОТЧЁТ
к лабораторной работе №2
на тему
**«ПОСТРОЕНИЕ И ИСПОЛЬЗОВАНИЕ КОРПУСОВ ТЕКСТОВ
ЕСТЕСТВЕННОГО ЯЗЫКА»**

Выполнил студент группы 321701
СЕМЕНЯКО Владимир Дмитриевич

(дата, подпись студента)

Проверил
КРАПИВИН Юрий Борисович

(дата, подпись преподавателя)

Минск 2026

1 ЗАДАНИЕ

Спроектировать и программно реализовать структуры хранения данных и алгоритмы их обработки, необходимые в рамках базовых требований к разрабатываемому корпусному менеджеру.

Вариант 21:

- язык текста — английский;
- предметная область — музыка (музыкальные жанры, инструменты, концертная жизнь);
- форматы входных документов — TXT, RTF, PDF, DOC, DOCX;
- выходные данные — частотные характеристики словоформ, лемм и грамматических категорий, лемма и её морфологические характеристики, конкордансные списки, метаданные документов.

Необходимо:

- сформировать электронный корпус текстов по выбранной предметной области;
- разработать корпусный менеджер с графическим интерфейсом, обеспечивающий:
 - добавление документов в корпус и их морфологическую разметку;
 - сохранение и загрузку корпуса из файла;
 - поиск по корпусу (по лемме и по словоформе);
 - получение частотных характеристик словоформ, лемм и частей речи (POS);
 - получение конкордансов;
 - просмотр морфологических характеристик выбранной леммы;
 - просмотр и фильтрацию документов по метаданным.

2 ОПИСАНИЕ СТРУКТУР ХРАНЕНИЯ ДАННЫХ И АЛГОРИТМОВ ОБРАБОТКИ

2.1 Структуры данных

В разработанном приложении используются следующие основные структуры данных.

- Текст документа — строка (`str`), результат извлечения текста из файла (TXT, RTF, PDF, DOC, DOCX). Кодировка UTF-8.
- Список токенов с морфологической информацией — список словарей `list[dict]`, каждая запись имеет вид `{"word", "lemma", "pos"}`.

Список получается после токенизации, теггинга частей речи и лемматизации с использованием библиотеки NLTK (функция `tokenize_with_pos` из лабораторной работы №1).

– Список документов корпуса `documents` — список словарей:

```
{
    "id": int,                % внутренний идентификатор
    "filepath": str,          % полный путь к файлу
    "title": str,             % название
    "author": str,            % автор (при наличии)
    "year": str,              % год (при наличии)
    "source": str,           % источник (при наличии)
    "text": str,              % полный текст документа
    "tokens": list[dict]      % список с word, lemma, pos
}
```

– Индексы корпуса:

– индекс словоформ `_word_index` — для каждой словоформы хранится список пар (`doc_idx`, `token_idx`), указывающих на позицию в конкретном документе;

– индекс лемм `_lemma_index` — аналогично, но по лемме;

– индекс частей речи `_pos_index`: — для каждой части речи (POS-тега) хранится список вхождений.

– Частотные словари:

– частоты словоформ — `dict[str, int]`, формируется при вызове метода `get_word_form_frequencies`;

– частоты лемм — `dict[str, int]`, формируется при вызове метода `get_lemma_frequencies`;

– частоты грамматических категорий (POS) — `dict[str, int]`, формируется при вызове метода `get_pos_frequencies`.

– Описание морфологии леммы — список записей, возвращаемый методом `get_morphology_for_lemma`.

– Метаданные документа — словарь с полями `title`, `author`, `year`, `source`, `filepath`, `tokens_count`.

– Сохранённый корпус (JSON-файл) — объект с ключами `name`, `"next_id"`, `"documents"`, где список `"documents"` содержит объекты указанной выше структуры. Файл используется для переноса и восстановления корпуса между сессиями работы.

2.2 Структурно-функциональная схема разработанного приложения

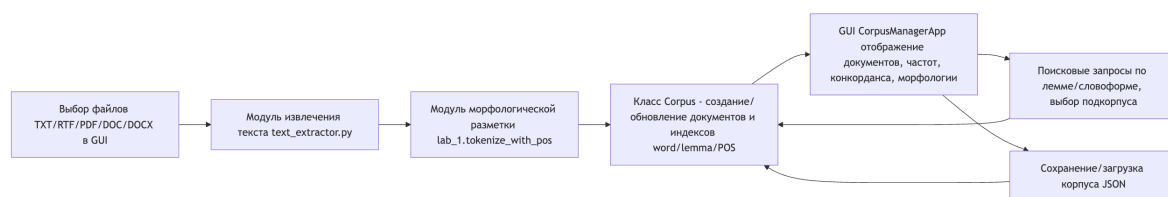


Рисунок 1 – Общий поток работы корпусного менеджера



Рисунок 2 – Алгоритм построения словаря сочетаемости

2.3 Алгоритмы обработки (базовые требования)

Извлечение текста из файлов. В зависимости от расширения файла вызывается соответствующая ветка модуля `text_extractor.py`:

- TXT — чтение текстового файла с попыткой декодирования в кодировках UTF-8, cp1252, latin-1; при ошибке используется `errors="replace"`;
- RTF — чтение бинарного файла, декодирование в строку и преобразование в обычный текст с помощью функции `rtf_to_text` из библиотеки `striprtf`;
- PDF — использование библиотеки `pypdf`: чтение документа, последовательный обход страниц, извлечение текста методом `page.extract_text()`;
- DOC/DOCX — повторное использование функции `extract_text` из лабораторной работы №1 (библиотека `python-docx` для DOCX и упрощённый парсер для DOC).

Морфологическая разметка. После извлечения текста вызывается функция `tokenize_with_pos` (лабораторная работа №1). Алгоритм:

- токенизация текста (`nltk.word_tokenize`) с приведением к нижнему регистру;
- отбор буквенных токенов длиной более одного символа;
- теггинг частей речи (`nltk.pos_tag`, теги Penn Treebank);
- приведение словоформ к леммам с помощью `WordNetLemmatizer` с учётом POS-тегов;
- формирование списка токенов `{"word", "lemma", "pos"}`.

Построение корпуса и индексов. Метод `add_document` класса `Corpus` выполняет:

- проверку наличия текста и его длины;
- генерацию идентификатора документа и формирование структуры документа (метаданные, полный текст, список токенов);
- добавление документа в список `documents`;
- для каждого токена добавление ссылок в индексы `_word_index`, `_lemma_index`, `_pos_index`.

При удалении документа (метод `remove_document`) корпус пересобирает индексы, проходя по текущему списку документов.

Поиск и конкорданс. Поиск реализован методом `search`:

- при поиске по одному слову выбирается соответствующий индекс (`_lemma_index` или `_word_index`), возвращаются все вхождения;
- при поиске по фразе сначала отбираются кандидаты по первому слову, затем для каждого кандидата проверяется совпадение всей последовательности лемм или словоформ.

Метод `get_concordance` строит конкордансные строки:

- для каждого вхождения выделяется окно заданного размера слева и справа;
- формируются строки вида «левый контекст — целевое слово — правый контекст» с указанием документа.

Частотные характеристики. Методы `get_word_form_frequencies`, `get_lemma_frequencies`, `get_pos_frequencies` обходят выбранные документы и подсчитывают частоты словоформ, лемм и POS-тегов. Результаты сортируются по убыванию частоты и используются для заполнения соответствующих вкладок GUI.

Морфология леммы. Метод `get_morphology_for_lemma`:

- по индексу лемм находит все вхождения заданной леммы;
- накапливает частоты по парам (словоформа, POS);
- возвращает отсортированный список записей с указанием словоформы, тега POS, его текстового описания и частоты.

Графический интерфейс. Графический интерфейс реализован на `tkinter` в модуле `app_gui.py` и включает:

- вкладку «Документы корпуса» — просмотр списка документов, фильтрация по выбранным, удаление, просмотр полного текста (двойной щелчок);
- вкладку «Частотные характеристики» — три под-вкладки для словоформ, лемм и частей речи;
- вкладку «Конкорданс» — таблица контекстов вхождений запроса;
- вкладку «Морфология леммы» — таблица словоформ и частот для введенной леммы;
- вкладку «Метаданные» — вывод библиографической информации по выбранному документу;
- панель поиска с выбором типа запроса (по лемме / по словоформе) и размера контекста.

3 ОЦЕНКА БЫСТРОДЕЙСТВИЯ ПРИЛОЖЕНИЯ

Для оценки быстродействия были подготовлены пять наборов тестовых документов в папке `data/`:

- базовые музыкальные тексты:
- `music_genres.txt` (жанры музыки, 894 символа),
- `instruments.txt` (музыкальные инструменты, 861 символ),
- `concert_life.txt` (концертная жизнь, 860 символов);
- дополнительные бенчмарки:
- `bench_1_short.txt` (короткий текст, 393 символа),
- `bench_2_short.txt` (короткий текст, 402 символа),
- `bench_3_medium.txt` (средний текст, 874 символа),
- `bench_4_medium.txt` (средний текст, 873 символа),
- `bench_5_long.txt` (длинный текст, 1900 символов).

Тесты проводятся следующим образом (во всех случаях документы добавляются через кнопку «Добавить документы» в GUI):

- **Тест 1** — загрузить только `music_genres.txt`;
- **Тест 2** — загрузить файл из теста 1, а также `instruments.txt`, `concert_life.txt`;
- **Тест 3** — загрузить файлы теста 2 и дополнительно `bench_1.txt`;
- **Тест 4** — загрузить файлы теста 3 и дополнительно `bench_2.txt` и `bench_3.txt`;
- **Тест 5** — загрузить файлы теста 4 и дополнительно `bench_4.txt` и `bench_5.txt`.

3.1 Оценка времени построения корпуса

Таблица 3.1 – Оценка быстродействия

Номер теста	Количество документов	Объем текста	Время построения корпуса
1	1	894	1.38с
2	3	2615	1.88с
3	4	3008	1.96с
4	6	4284	2.38с
5	8	7057	2.69с

3.2 Время выполнения типовых запросов по корпусу

Таблица 3.2 – Оценка быстродействия

Номер теста	Поиск по лемме music	Конкорданс по concert	Обновление частот	Морфология леммы instrument
1	0.00002	0.00004	0.00194	0.00012
2	0.00002	0.00073	0.00419	0.00014
3	0.00003	0.00050	0.00419	0.00019
4	0.00018	0.00017	0.00886	0.00014
5	0.0002	0.00029	0.01002	0.00055

ВЫВОД

– Разработано приложение-корпусный менеджер для текстов на английском языке по предметной области «Музыка».

– Реализованы требования задания: поддержка форматов TXT, RTF, PDF, DOC, DOCX; построение корпуса с морфологической разметкой; получение частотных характеристик, конкордансов, морфологии лемм и мета-

данных документов; графический интерфейс и система помощи.

- Для морфологического анализа и токенизации использованы результаты лабораторной работы №1 (NLTK, лемматизация и POS-теггинг), что обеспечивает переиспользование кода и единый подход к обработке текста.

- Быстродействие приложения является приемлемым для корпусов учебного и среднего объёма; при увеличении объёма данных возможно масштабирование за счёт оптимизации структур индексов и использования более эффективных инструментов хранения.